Section A – Debugging Questions

1. The Diamond Problem in Multipath Hybrid Inheritance

1. Compilation error:

The statement ta.idNumber causes an ambiguity error because TeachingAssistant inherits two separate copies of Person, one through Student and another through Employee.

Error:

error: request for member 'idNumber' is ambiguous

2. Memory layout:

Without virtual inheritance, the object contains two separate Person subobjects.

```
    TeachingAssistant
      /       \
    Student    Employee
      |          |
     Person      Person
    idNumber    idNumber
      101         202
```

Therefore, the compiler cannot decide which idNumber should be accessed.

3. Corrected code using virtual inheritance:

```
#include <iostream>
using namespace std;

class Person {
public:
   int idNumber;

   Person(int id = 0) {
      idNumber = id;
      cout << "Person initialized: " << idNumber << endl;
   }
};

class Student : virtual public Person {
```

```
public:
   Student() {}
};

class Employee : virtual public Person {
public:
   Employee() {}
};

class TeachingAssistant : public Student, public Employee {
public:
   TeachingAssistant(int id) : Person(id) {}
};

int main() {
   TeachingAssistant ta(101);
   cout << "TA ID: " << ta.idNumber << endl;
   return 0;
}
```

4. Constructor delegation:

- Virtual inheritance creates only one shared Person object.
- The most-derived class, TeachingAssistant, is responsible for initializing the virtual base class.
- Therefore, Person(id) is called directly in the TeachingAssistant initializer list.

Output:

```
Person initialized: 101
TA ID: 101
```

2. Missing Base Constructor in Derived Initializer List

1. Reason for compilation error:

The Device class has only a parameterized constructor and does not have a default constructor.

When a Sensor object is created, the compiler tries to call Device() automatically. Since it does not exist, compilation fails before the constructor body executes.

2. Constructor execution order:

1. Base class constructor (Device) executes first.
2. Derived class constructor (Sensor) executes next.
3. The derived class constructor body executes after base initialization.

3. Corrected code:

```
#include <iostream>
#include <string>
using namespace std;

class Device {
protected:
   string macAddress;

public:
   Device(string mac) : macAddress(mac) {
      cout << "Device configured with MAC: "
           << macAddress << endl;
   }
};

class Sensor : public Device {
private:
   int samplingRate;

public:
   Sensor(string mac, int rate)
      : Device(mac), samplingRate(rate) {
   }

   void display() {
      cout << "MAC: " << macAddress
           << ", Rate: " << samplingRate
           << " Hz" << endl;
   }
};

int main() {
   Sensor tempSensor("00:1A:2B:3C:4D:5E", 100);
   tempSensor.display();
   return 0;
}
```

4. Output:

Device configured with MAC: 00:1A:2B:3C:4D:5E
MAC: 00:1A:2B:3C:4D:5E, Rate: 100 Hz

3. Base Class Pointer Member Access & Object Slicing

1. Reason for compilation error:

vPtr is declared as a Vehicle* pointer. Therefore, it can access only the members declared in the Vehicle class.

Although it points to a SportsCar object, engageTurbo() belongs to SportsCar. So, calling it through vPtr causes a compilation error.

2. Object slicing:

Object slicing happens when a derived class object is copied into a base class object by value.

Vehicle plainVehicle = car;

Only the Vehicle part is copied. The derived member turboEnabled is lost.

3. Correct method:

- Use downcasting to access the derived class method.
- Use a base class reference or pointer to avoid object slicing.

4. Corrected code:

```
#include <iostream>
using namespace std;

class Vehicle {
public:
   int maxSpeed;

   Vehicle(int s) : maxSpeed(s) {}

   void honk() {
      cout << "General Horn" << endl;
   }

   virtual ~Vehicle() {}
};
```

```
class SportsCar : public Vehicle {
public:
   bool turboEnabled;

   SportsCar(int s, bool turbo)
      : Vehicle(s), turboEnabled(turbo) {}

   void engageTurbo() {
      cout << "Turbo Engaged!" << endl;
   }
};

int main() {
   Vehicle *vPtr = new SportsCar(300, true);

   vPtr->honk();

   SportsCar *sPtr = static_cast<SportsCar*>(vPtr);
   sPtr->engageTurbo();

   SportsCar car(250, true);
   Vehicle &plainVehicle = car;

   cout << "Max Speed: " << plainVehicle.maxSpeed << endl;

   delete vPtr;
   return 0;
}
```

Output:

```
General Horn
Turbo Engaged!
Max Speed: 250
```

Note: static_cast is valid here because the pointer is known to point to a SportsCar. A checked dynamic_cast would be safer when the actual derived type is uncertain.

4. Incorrect Return Type in this Pointer Method Chaining

1. Reason for error:

The setWidth() method returns a pointer (WindowBox*), but the next method uses the dot operator.

The dot operator works with objects and references, not pointers. Therefore, chaining fails.

2. Difference between this and *this:

this	*this
Pointer to the current object	Current object itself
Type is WindowBox*	Type is WindowBox& when used as a reference
Returns an address	Refers to the actual object

3. Correction:

Change the return type to WindowBox& and return *this.

4. Corrected code:

```
#include <iostream>
using namespace std;

class WindowBox {
private:
   int width, height;

public:
   WindowBox() : width(0), height(0) {}

   WindowBox& setWidth(int width) {
      this->width = width;
      return *this;
   }

   WindowBox& setHeight(int height) {
      this->height = height;
      return *this;
   }
```

```
    void display() {
        cout << "Window: " << width
             << " x " << height << endl;
    }
};

int main() {
    WindowBox box;

    box.setWidth(800).setHeight(600);
    box.display();

    return 0;
}
```

Output:

Window: 800 x 600

5. Private Inheritance Access Violations & Inaccessible Base Cast

1. Explanation:

When SecureChannel privately inherits RawSocket, the public members of RawSocket become private inside SecureChannel.

- openPort() can be accessed inside SecureChannel.
- closePort() cannot be accessed directly from main().

2. Reason for upcasting error:

Private inheritance makes the base class inaccessible outside the derived class.

Therefore, this statement causes an error:

RawSocket *socketPtr = &channel;

Upcasting is allowed when the base class is publicly accessible.

3. Two ways to make closePort() accessible:

- Approach A: Use using RawSocket::closePort; inside the public section of SecureChannel.
- Approach B: Use public inheritance or composition.

4. Corrected code using Approach A:

```
#include <iostream>
using namespace std;

class RawSocket {
public:
   void openPort(int port) {
      cout << "Port " << port << " opened." << endl;
   }

   void closePort() {
      cout << "Port closed." << endl;
   }
};

class SecureChannel : private RawSocket {
public:
   using RawSocket::closePort;

   void establishEncryptedTunnel(int port) {
      openPort(port);
      cout << "Tunnel encrypted with TLS 1.3" << endl;
   }
};

int main() {
   SecureChannel channel;

   channel.establishEncryptedTunnel(443);
   channel.closePort();

   return 0;
}
```

Output:

```
Port 443 opened.
Tunnel encrypted with TLS 1.3
Port closed.
```

Note: Approach A makes closePort() accessible, but private inheritance still prevents an ordinary external upcast to RawSocket*.

Section B – Pseudocode Writing Questions

1. University ERP Hybrid Hierarchy with Virtual Base Classes

1. Structured pseudocode:

```
CLASS Person
   DECLARE personID AS INTEGER
   DECLARE fullName AS STRING
   DECLARE email AS STRING

   CONSTRUCTOR(id, name, mail)
      personID <- id
      fullName <- name
      email <- mail
   END CONSTRUCTOR
END CLASS


CLASS Student INHERITS VIRTUAL Person
   DECLARE gpa AS REAL
   DECLARE creditsEarned AS INTEGER

   CONSTRUCTOR(g, credits)
      gpa <- g
      creditsEarned <- credits
   END CONSTRUCTOR
END CLASS


CLASS Faculty INHERITS VIRTUAL Person
   DECLARE academicDepartment AS STRING
   DECLARE rankTitle AS STRING

   CONSTRUCTOR(dept, rank)
      academicDepartment <- dept
      rankTitle <- rank
   END CONSTRUCTOR
END CLASS
```

```
CLASS TeachingAssistant INHERITS Student, Faculty
   DECLARE stipendAmount AS REAL
   DECLARE assignedCourse AS STRING

   CONSTRUCTOR(id, name, mail, g, credits,
            dept, rank, stipend, course)
      CONSTRUCTOR Person(id, name, mail)
      CONSTRUCTOR Student(g, credits)
      CONSTRUCTOR Faculty(dept, rank)

      stipendAmount <- stipend
      assignedCourse <- course
   END CONSTRUCTOR

   METHOD DisplayTARecord()
      PRINT personID
      PRINT fullName
      PRINT email
      PRINT gpa
      PRINT academicDepartment
      PRINT stipendAmount
      PRINT assignedCourse
   END METHOD
END CLASS


BEGIN MAIN
   DECLARE ta AS TeachingAssistant

   ta <- NEW TeachingAssistant(
      101, "Arun", "arun@gmail.com",
      8.5, 90, "Computer Science",
      "Assistant Professor", 15000, "C++"
   )

   CALL ta.DisplayTARecord()
END MAIN
```

2. Dry run:

Attribute	**Value**

Person ID	101
Name	Arun
Email	arun@gmail.com
GPA	8.5
Department	Computer Science
Stipend	15000
Course	C++

Output:

Person ID: 101
Name: Arun
Email: arun@gmail.com
GPA: 8.5
Department: Computer Science
Stipend: 15000
Course: C++

Explanation: Virtual inheritance ensures that only one shared Person object exists. The TeachingAssistant constructor initializes this virtual base class.

2. Polymorphic Payroll with Abstract Base Class & Base Pointers

1. Structured pseudocode:

```
ABSTRACT CLASS Employee
   DECLARE employeeID AS STRING
   DECLARE name AS STRING

   CONSTRUCTOR(id, n)
      employeeID <- id
      name <- n
   END CONSTRUCTOR
```

```
    ABSTRACT METHOD CalculatePay() RETURNS REAL

    METHOD DisplayHeader()
        PRINT employeeID, name
    END METHOD
END CLASS


CLASS SalariedEmployee EXTENDS Employee
    DECLARE monthlySalary AS REAL

    CONSTRUCTOR(id, n, salary)
        CONSTRUCTOR Employee(id, n)
        monthlySalary <- salary
    END CONSTRUCTOR

    METHOD CalculatePay() RETURNS REAL
        RETURN monthlySalary
    END METHOD
END CLASS


CLASS HourlyEmployee EXTENDS Employee
    DECLARE hourlyWage AS REAL
    DECLARE hoursWorked AS INTEGER

    CONSTRUCTOR(id, n, wage, hours)
        CONSTRUCTOR Employee(id, n)
        hourlyWage <- wage
        hoursWorked <- hours
    END CONSTRUCTOR

    METHOD CalculatePay() RETURNS REAL
        IF hoursWorked <= 40 THEN
            RETURN hourlyWage * hoursWorked
        ELSE
            RETURN (40 * hourlyWage) +
                ((hoursWorked - 40) * hourlyWage * 1.5)
        END IF
    END METHOD
END CLASS
```

```
BEGIN MAIN
   DECLARE employees AS ARRAY OF POINTER TO Employee
   DECLARE total AS REAL <- 0

   employees[0] <- NEW SalariedEmployee(
      "E101", "Arun", 8500
   )

   employees[1] <- NEW HourlyEmployee(
      "E102", "Bala", 50, 48
   )

   FOR i <- 0 TO 1
      CALL employees[i]->DisplayHeader()
      pay <- employees[i]->CalculatePay()

      PRINT "Pay:", pay
      total <- total + pay
   END FOR

   PRINT "Total Disbursement:", total

   DELETE employees[0]
   DELETE employees[1]
END MAIN
```

2. Dry-run calculations:

Salaried employee:

\[Pay = 8500 \]

Hourly employee:

- Regular pay = 40 × 50 = 2000
- Overtime hours = 48 − 40 = 8
- Overtime pay = 8 × 50 × 1.5 = 600
- Total pay = 2000 + 600 = 2600

Dry-run table:

Employee	Calculation	Pay

Salaried	Monthly salary	8500
Hourly	2000 + 600	2600
Total	8500 + 2600	11100

Output:

Employee: E101 Arun
Pay: 8500

Employee: E102 Bala
Pay: 2600

Total Disbursement: 11100

Explanation: The Employee class is abstract because it contains a pure virtual function. Base class pointers call the appropriate derived class CalculatePay() method at runtime.

3. Composite Canvas Engine with Member Classes & this Chaining

1. Structured pseudocode:

```
CLASS Point2D
   DECLARE x, y AS REAL

   CONSTRUCTOR(newX = 0.0, newY = 0.0)
      x <- newX
      y <- newY
   END CONSTRUCTOR
END CLASS


CLASS Circle
   DECLARE centerPoint AS Point2D
   DECLARE radius AS REAL

   CONSTRUCTOR()
      centerPoint <- NEW Point2D(0.0, 0.0)
      radius <- 1.0
   END CONSTRUCTOR
```

```
    METHOD SetCenter(newX, newY) RETURNS REF Circle
        centerPoint.x <- newX
        centerPoint.y <- newY
        RETURN REF THIS
    END METHOD

    METHOD SetRadius(r) RETURNS REF Circle
        IF r > 0 THEN
            radius <- r
        ELSE
            PRINT "Invalid radius"
        END IF

        RETURN REF THIS
    END METHOD

    METHOD Draw()
        PRINT "Center:", centerPoint.x, centerPoint.y
        PRINT "Radius:", radius
    END METHOD
END CLASS


BEGIN MAIN
    DECLARE myCircle AS Circle

    CALL myCircle.SetCenter(12.5, 30.0)
            .SetRadius(7.5)
            .Draw()
END MAIN
```

2. Dry run:

Step	Operation	Center	Radius
1	Create Circle	(0, 0)	1.0
2	SetCenter(12.5, 30.0)	(12.5, 30.0)	1.0
3	SetRadius(7.5)	(12.5, 30.0)	7.5

4	Draw()	(12.5, 30.0)	7.5

Output:

Center: 12.5, 30
Radius: 7.5

Explanation:

- Circle contains a Point2D object. This is called composition or a has-a relationship.
- RETURN REF THIS returns the current object by reference.
- This allows multiple methods to be chained using the dot operator.

4. Multi-Level Autonomous Fleet Telemetry with Base Pointers

1. Structured pseudocode:

```
CLASS Vehicle
   DECLARE vehicleVIN AS STRING
   DECLARE baseWeightKg AS REAL

   CONSTRUCTOR(vin, weight)
      vehicleVIN <- vin
      baseWeightKg <- weight
   END CONSTRUCTOR

   METHOD DisplaySpecs()
      PRINT "VIN:", vehicleVIN
      PRINT "Base Weight:", baseWeightKg
   END METHOD
END CLASS


CLASS AutonomousVehicle EXTENDS Vehicle
   DECLARE lidarSensorID AS STRING
   DECLARE firmwareVersion AS STRING

   CONSTRUCTOR(vin, weight, lidar, firmware)
      CONSTRUCTOR Vehicle(vin, weight)
      lidarSensorID <- lidar
      firmwareVersion <- firmware
   END CONSTRUCTOR
END CLASS
```

```
CLASS DeliveryDrone EXTENDS AutonomousVehicle
   DECLARE payloadCapacityKg AS REAL
   DECLARE batteryChargePercent AS REAL

   CONSTRUCTOR(vin, weight, lidar, firmware,
               payload, battery)
      CONSTRUCTOR AutonomousVehicle(
         vin, weight, lidar, firmware
      )

      payloadCapacityKg <- payload
      batteryChargePercent <- battery
   END CONSTRUCTOR

   METHOD DisplayFlightStatus()
      CALL DisplaySpecs()
      PRINT "Lidar ID:", lidarSensorID
      PRINT "Firmware:", firmwareVersion
      PRINT "Payload Capacity:", payloadCapacityKg
      PRINT "Battery:", batteryChargePercent
   END METHOD
END CLASS


BEGIN MAIN
   DECLARE vPtr AS POINTER TO Vehicle
   DECLARE dPtr AS POINTER TO DeliveryDrone

   dPtr <- NEW DeliveryDrone(
      "VIN101", 12.5, "LIDAR01",
      "V2.0", 5.0, 90
   )

   vPtr <- dPtr

   CALL vPtr->DisplaySpecs()

   dPtr <- CAST vPtr TO POINTER TO DeliveryDrone

   CALL dPtr->DisplayFlightStatus()
```

```
    DELETE dPtr
END MAIN
```

2. Constructor invocation order:

When a DeliveryDrone object is created, constructors execute from the root base class to the most-derived class.

1. Vehicle constructor executes.
2. AutonomousVehicle constructor executes.
3. DeliveryDrone constructor executes.

Dry-run table:

Class	Initialized data
Vehicle	VIN101, 12.5 kg
AutonomousVehicle	LIDAR01, V2.0
DeliveryDrone	5.0 kg, 90%

Output:

```
VIN: VIN101
Base Weight: 12.5

VIN: VIN101
Base Weight: 12.5
Lidar ID: LIDAR01
Firmware: V2.0
Payload Capacity: 5
Battery: 90
```

Explanation:

- This is multilevel inheritance.
- A Vehicle* pointer can point to a DeliveryDrone object through upcasting.
- The derived pointer is used to access drone-specific information.
- The dynamically allocated object is deleted after use.

5. Dynamic Object Pointer Array & Zero-Copy Sorting Engine

1. Structured pseudocode:

```
CLASS Product
   DECLARE sku AS STRING
   DECLARE price AS REAL
   DECLARE stockCount AS INTEGER

   CONSTRUCTOR(s, p, stock)
      sku <- s
      price <- p
      stockCount <- stock
   END CONSTRUCTOR

   METHOD PrintProduct()
      PRINT sku, price, stockCount
   END METHOD
END CLASS


BEGIN MAIN
   DECLARE catalog AS ARRAY OF POINTER TO Product
   DECLARE temp AS POINTER TO Product
   DECLARE N <- 3

   catalog[0] <- NEW Product("Laptop", 1200, 10)
   catalog[1] <- NEW Product("Mouse", 25, 50)
   catalog[2] <- NEW Product("Monitor", 350, 20)

   FOR i <- 0 TO N-2
      FOR j <- 0 TO N-i-2

         IF catalog[j]->price > catalog[j+1]->price THEN

            temp <- catalog[j]
            catalog[j] <- catalog[j+1]
            catalog[j+1] <- temp

         END IF

      END FOR
   END FOR
```

```
    PRINT "Products in ascending order:"

    FOR i <- 0 TO N-1
        CALL catalog[i]->PrintProduct()
    END FOR

    FOR i <- 0 TO N-1
        DELETE catalog[i]
    END FOR
END MAIN
```

2. Dry-run trace table:

Initial order:

Position	Product	Price
0	Laptop	1200
1	Mouse	25
2	Monitor	350

Pass 1:

- Compare Laptop (1200) and Mouse (25): swap pointers.
- Compare Laptop (1200) and Monitor (350): swap pointers.

Position	Product after Pass 1	Price
0	Mouse	25
1	Monitor	350
2	Laptop	1200

Pass 2:

- Compare Mouse (25) and Monitor (350): no swap.

Position	Final Product	Price
0	Mouse	25
1	Monitor	350
2	Laptop	1200

Final output:

Products in ascending order:

Mouse 25 50
Monitor 350 20
Laptop 1200 10